

TinyTorch Activation Benchmark

Bugs, Root Causes, Solutions & Benchmark Lessons

This document records the debugging process for the TinyTorch activation-performance benchmark. It focuses on the callable-module error, benchmark methodology improvements, and the final measured results.

Final benchmark: 1,000,000 float32 elements, 10 timing runs per activation.

1. Bug #1 — Activation Object Is Not Callable

The benchmark failed with the following exception:

```
TypeError: 'Sigmoid' object is not callable
```

The failing line was `sigmoid(test_data)`. Python treats this syntax as a call to `sigmoid.__call__(test_data)`. The Sigmoid object did not provide a `__call__` method, so Python raised a `TypeError`.

Root Cause

The activation classes exposed a forward-style interface, while the benchmark expected framework modules to behave like callable functions.

Solution A — Immediate Benchmark Fix

If the activation classes implement `forward()`, call that method directly:

```
_ = relu.forward(test_data)
_ = sigmoid.forward(test_data)
_ = tanh.forward(test_data)
_ = gelu.forward(test_data)
```

Solution B — Recommended TinyTorch Architecture

A cleaner framework design is to make the base `Module` callable and route calls to `forward()`. This keeps the public API simple and consistent across all modules.

```
class Module:
    def __call__(self, *args, **kwargs):
        return self.forward(*args, **kwargs)

    def forward(self, *args, **kwargs):
        raise NotImplementedError
```

With this design, `sigmoid(test_data)` becomes equivalent to calling `sigmoid.forward(test_data)`. This is the preferred solution for TinyTorch because it gives all modules a consistent callable interface.

2. Bug #2 — Incomplete Warm-Up

The original benchmark warmed up only ReLU and Sigmoid before timing all four activations. Tanh and GELU were timed without the same warm-up treatment.

Original pattern

```
_ = relu(test_data)
_ = sigmoid(test_data)
```

Solution

Warm up every operation that will be benchmarked:

```
_ = relu(test_data)
_ = sigmoid(test_data)
```

```
_ = tanh(test_data)
_ = gelu(test_data)
```

This makes the benchmark methodology more consistent and reduces the chance that initialization or first-use overhead contaminates one measurement.

3. Improvement — Use a High-Resolution Timer

The original benchmark used `time.time()`. It works, but `time.perf_counter()` is more appropriate for measuring short-duration performance because it is designed for elapsed-time measurements.

```
start = time.perf_counter()
for _ in range(n_runs):
    _ = sigmoid(test_data)
elapsed = time.perf_counter() - start
```

4. Final Benchmark Results

Activation	Time / 1M elements	Relative to ReLU
ReLU	7.74 ms	1.0×
Tanh	15.86 ms	2.0×
Sigmoid	56.35 ms	7.3×
GELU	67.39 ms	8.7×

Observed result: In this TinyTorch implementation and test environment, Sigmoid took about 7.3× as long as ReLU, while GELU took about 8.7× as long. Tanh was about 2.0× the ReLU baseline.

5. What the Benchmark Demonstrates

- **ReLU:** very simple element-wise computation, making it an efficient baseline.
- **Tanh and Sigmoid:** involve more expensive mathematical operations such as exponential evaluation.
- **GELU:** the current implementation is the most expensive of the four in this benchmark.
- **Scaling matters:** when activations are evaluated over very large tensors or repeated many times, per-element computational differences can accumulate.

6. Important Interpretation Note

These timings are measurements of the **current TinyTorch implementation on the current machine and runtime**. They should not be treated as universal hardware-independent costs. Optimized libraries, CPU vectorization, compiler behavior, memory layout, approximation methods, and hardware can substantially change absolute timings.

Also, activation choice is not determined by speed alone. GELU, Sigmoid, Tanh, and ReLU have different mathematical behavior and are selected according to the architecture and learning objective.

7. Recommended Next Experiment

To turn this benchmark into a stronger TinyTorch systems experiment, repeat it across multiple tensor sizes:

1,000	10,000	100,000	1,000,000	10,000,000
-------	--------	---------	-----------	------------

Plot execution time against tensor size and investigate whether each implementation scales approximately linearly with the number of elements. A second plot can show relative cost compared with ReLU.

8. Clean Benchmark Checklist

- All activation objects use the same public calling convention.
- Every activation is warmed up before measurement.
- `time.perf_counter()` is used for elapsed-time measurements.
- The same input size and dtype are used for every activation.
- Multiple runs are averaged.
- Relative performance is reported against a clearly defined baseline.
- Results are labeled as measurements from the current implementation/environment.

TinyTorch takeaway: debugging the callable-module error exposed an important framework-design principle — a consistent Module API makes both model code and performance experiments easier to write, test, and reason about.